THE ODIN PROJECT

# Mastering Intermediate HTML & CSS

A practical walkthrough of the curriculum

SVGs & Tables • Advanced CSS • Forms & Validation • CSS Grid

Presenter: [Your Name]

# Course Overview & Objectives

The bridge between beginner foundations and more complex, responsive web projects.

### 01 Intermediate HTML

Emmet, scalable vector graphics (SVG), and semantic tables.

### 02 Intermediate CSS

The cascade, units, selectors, positioning, and visual styling.

### 03 Forms & Validation

Accessible form structure and built-in browser validation.

### 04 CSS Grid

Two-dimensional layouts, applied in an admin dashboard.

**By the end: write clearer HTML, control CSS layouts, and build more usable interfaces.**

# Module 1 — Intermediate HTML Concepts

### Emmet

Use shorthand in editors such as VS Code to generate repetitive HTML quickly.

### SVG graphics

Vector-based images stay sharp when scaled. Great for icons, logos, and diagrams.

### Semantic tables

Use table elements to represent data in rows and columns—not to lay out a page.

## Example: a semantic table

```
<table>
  <thead>
    <tr><th>Name</th><th>Role</th></tr>
  </thead>
  <tbody>
    <tr><td>Ana</td><td>Developer</td></tr>
  </tbody>
</table>
```

**Key tags: <table>, <thead>, <tbody>, <tr>, <th>, <td>**

# Module 2 — Intermediate CSS Concepts

## Cascade & specificity

Understand why one rule wins over another and how browser defaults affect elements.

## CSS units

px for fixed sizing; rem/em for relative sizing; vw/vh for viewport dimensions.

## Selectors & positioning

Use attribute/combinator selectors and understand relative, absolute, fixed, and sticky.

## Visual polish

Use gradients, background images, CSS variables, and cross-browser testing.

```
:root { --brand: #2b6a73; }
.card {
  color: var(--brand);
  padding: 1rem;
  position: relative;
}
```

**Tip: reusable variables help keep a design consistent.**

# Module 3 — Forms & Validation

Goal: collect information clearly and help users fix mistakes before submitting.

### Structure

Use <form>, connected <label> elements, and suitable input types such as email and password.

### Validation

Use required, minlength, maxlength, and pattern to set browser-enforced rules.

### Accessibility

Clear labels, readable error messages, keyboard support, and good contrast matter.

## Example: built-in validation

```
<label for="email">Email</label>
<input id="email" name="email" type="email" required>

<label for="password">Password</label>
<input id="password" name="password" type="password" minlength="8" required>
```

**Project: Sign-up Form**

**Important: validate on the server too; browser checks alone are not security.**

# Module 4 — CSS Grid Layout

### Two-dimensional

Grid organizes content in rows and columns, making it useful for page-level layouts.

### Placement

Use grid-template-columns, grid-template-rows, and spanning to place content.

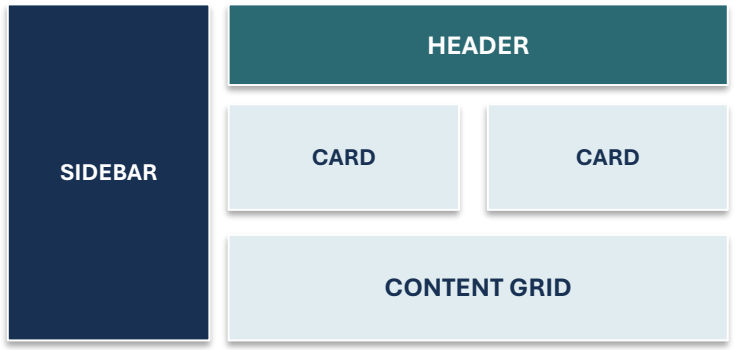
### Grid + Flexbox

Grid handles the larger structure; Flexbox aligns items within individual components.

## Example: dashboard layout

```
.dashboard {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: auto 1fr;
  min-height: 100vh;
}
@media (max-width: 700px) {
  .dashboard { grid-template-columns: 1fr; }
}
```

### Dashboard sketch



# Key Takeaways & Lessons Learned

- Semantic HTML makes content more meaningful and easier to maintain.
- Understanding the cascade, units, and positioning helps you debug CSS confidently.
- Use Flexbox for one-dimensional alignment and Grid for two-dimensional layouts.
- Accessible labels and clear validation improve form usability.
- Build iteratively and test at different viewport sizes and in different browsers.

**Practice turns concepts into skills: improve the Sign-up Form and Admin Dashboard projects.**

# Q&A

Questions or discussion?

### Live project showcase

Sign-up Form: [add your link]
Admin Dashboard: [add your link]

## Thank you for listening!